

Read the entire specification before you begin working on this project!
Pay particular attention to the rom.txt requirements.

0. Preliminaries

Honor Code Requirements

You must comply with all provisions of Virginia Tech's Honor System. Your Verilog code and submitted documentation must be your own work. Consider all information that you derive from your design process to be proprietary. Your design, coding, debugging, and testing must all derive from your own effort. It is an Honor Code Violation to share *any element of your design or implementation* with *any other person*, or to receive *any element of your work* from *any other person's design* – either from paper, from a computer file, or from the Internet. Review the AI Usage policy in the syllabus.

You may discuss general features of Quartus and the DE10-Lite board with other students. Direct all other questions about your design or implementation to 2544 course staff (instructor, GTA or ULA). Ask your instructor if you have any questions about what is and is not allowed under the Honor Code.

Objectives

After completing this assignment, you should be able to design, simulate, and implement an Arithmetic Circuit from a specification.

Learning Experience C Description

A function unit is a combinational circuit that performs a variety of arithmetic, logic, and shift operations. **Overall** in Learning Experience C (LE C.1.a, LE C.1.b, LE C.2), you will design a function unit that accepts two 8-bit 2s complement operands (OpA, OpB) and a 4-bit function select code (FS) as inputs and performs 13 different operations. The function unit will output the 8-bit result of the operation as well as four status bits (V, C, N, Z).

The function unit is comprised of multiple functional blocks. ***In this assignment (LE C.1.a), you will design and implement the functional block that performs six arithmetic operations.*** Three of those operations must be add (A+B), subtract (A-B) and mova (A). You may choose the remaining three operations from the list provided in this specification.

Preparation

Read this project specification completely before beginning any design or HDL coding. You must have access to a computer that can run the Quartus Prime Lite Edition software package. You will also need the DE10-Lite Board.

- Consult the appropriate sections of Chapter 3 and Chapter 8 of the textbook.
- Review the ECE2544 Lab Manual sections
 - “Switches, Buttons, and LEDs”,
 - “Restoring a Quartus Archive”,
 - “Simulating in Quartus and Qsim”,
 - “Troubleshooting Simulations”,
 - “Implementing Circuits on the DE10-Lite Board”
 - “Creating a Quartus Archive for Submission”
 - “Expectations For Transistor Counts and Propagation Delay Calculations” and
 - “Verilog File Expectations”.
- Consult Section 3 of the DE10-Lite User Manual for information on using the switches and LEDs.
- If you have any questions on the use of the DE10-Lite, see a member of the 2544 course staff.

1. Getting Started

1.1 Selecting the Operation Set

You will design and implement an arithmetic circuit that can perform six arithmetic operations on two 8-bit 2s complement operands, OpA and OpB. You must implement all three operations in the Required Operations table, plus three operations selected from the Elective Operations table below. The arithmetic circuit should generate an 8-bit result.

Table 1.1 Required Operations.

Operation name	Output	Description
add	$A + B$	Add A and B
sub	$A - B$	Subtract B from A
movA	A	No change to A

Table 1.2 Elective Operations. (Choose 3.)

Operation name	Output	Description
subBA	$B - A$	Subtract A from B
inc3A	$A + 3$	Add 3 to A
negA	$-A$	Negate A (2cm)
decA	$A - 1$	Subtract 1 from A
incB	$B + 1$	Add 1 to B
negB	$-B$	Negate B (2cm)
dec2B	$B - 2$	Subtract 2 from B

1.2 Navigating the Quartus Archive

1. Download the LE C.1.a Quartus archive and create a working folder for Learning Experience C.1.a (e.g. c:\ece2544\LE_C1a)

Tip: All your Quartus files for Learning Experience C.1.a, including waveform/simulation files must be stored in this working folder (not in sub folders).

2. Follow the directions in the lab manual to restore the archive **LE_C1a.qar**. When prompted, select the working folder you just created as the destination folder.

Caution: Once you have restored an archive, use the appropriate project file (*.qpf) to open the project in future instances. If you open the qar archive after restoring it for the first time, you could overwrite your saved work.

3. Open **LE_C1a.bdf**.

LE_C1a.bdf shows the top-level design. It contains two components: a ROM, which contains a set of 8-bit operands, and a block labeled **arith_circuit**. You will implement your arithmetic circuit design in the arith_circuit block. The rest of the system has been provided for you.

Right-click **arith_circuit** and choose **Open Design File** to open the Verilog model. Another way to do this is to choose Files in the Project Navigator, then double-click **arith_circuit.v**. The existing **arith_circuit.v** file contains a template you will fill in with your circuit design. The arithmetic circuit takes as input the two 8-bit operands A and B, and three of the switches on the DE10-Lite board (SW[9:7]). The switches control which of the six implemented

operations your arithmetic circuit will output at any one time. This means that as part of your design you will have to assign a 3-bit code to each operation.

The operands A and B will be supplied by a small Read Only Memory (ROM). The ROM contains eight 8-bit operands, which can be selected using the other switches on your DE10-Lite board. SW[2:0] select operand A and SW[5:3] select operand B, as described below.

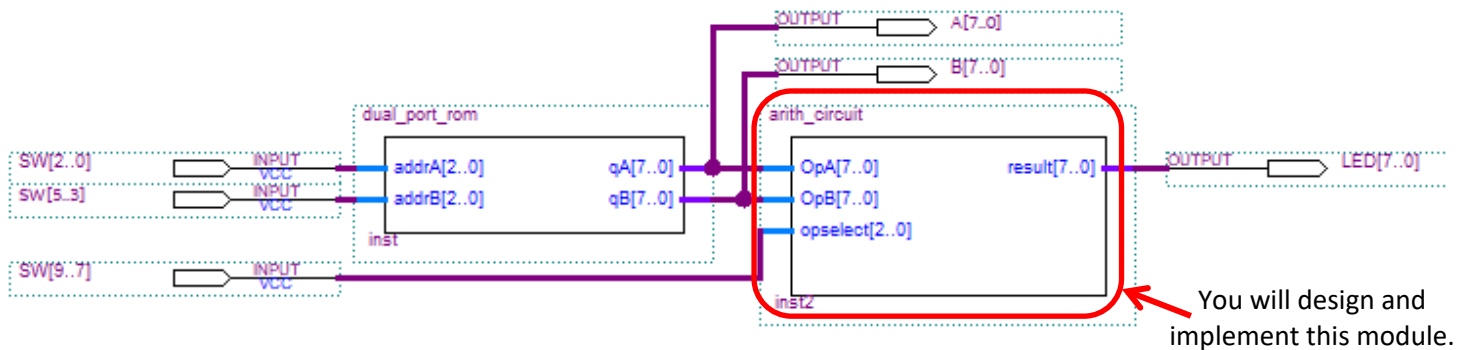


Figure 1.1. Schematic of the full system for Learning Experience C.1.a. Your responsibility is to implement the “arith_circuit” module. Do not modify any other .v or .bdf files.

4. Choose **Files** in the Project Navigator, then double-click **rom.txt**.

Each line of the text file has the hex value for one memory location. Since each word is 8 bits wide, each line of the file has a 2-digit hex number. For example, the 8-bit value in memory location 0 is 0x0F, or 0b00001111. You can select operand A from these eight values using SW[2:0]. You can select operand B from these eight values using SW[5:3]. For example, to select the contents of memory location 2 as operand A, set SW[2:0] to 010; to select the contents of memory location 6 as operand B, set SW[5:3] to 110.

You can modify the contents of the ROM by editing rom.txt. Be sure to recompile after editing.

Tip: This means operand A and operand B must be selected from data contained in the rom. You cannot input them directly, and they should not be assigned input values in simulations. To choose a specific operand data value, save it in the rom and use the switches to select it.

2. Circuit Design

Once you have selected your operation set, you must:

- Design a circuit that accepts 8-bit operands and a 3-bit operation select code, and correctly outputs the result of all six operations. You must consider efficient use of transistor resources in your design.
- Assign a 3-bit code to each operation that will be used to select that operation using SW[9:7]. Be sure to consider any necessary control circuits.
- Create and simulate a *Verilog model* in Quartus
- Implement and test your circuit design on your DE10-Lite Board.
- Prepare your files for submission.

2.1 Implementation

The Quartus archive project file posted for this project includes the schematic and Verilog files necessary to build the system. Do not modify any files except for the rom.txt file and the arith_circuit.v file. In particular, if you change the top-level schematic or modify the pin assignments, there is a chance you could damage your DE10-Lite board.

The port declaration for the `arith_circuit` module is as follows:

```
module arith_circuit (result, OpA, OpB, opselect);
input [2:0] opselect;
input [7:0] OpA, OpB;
output [7:0] result;
```

As shown on the schematic, the inputs `OpA` and `OpB` are supplied from the ROM output. The `opselect` input is connected to the DIP switches, `SW[9:7]`, and is used to select the operation. The output, `result`, is connected to the LEDs, with `LED[7]` being the most significant bit. This will allow you to test your circuit on the DE10-Lite board using the switches to select the operation and operands, and the LEDs to view the result.

As part of your design process, you will have to assign specific 3-bit code values for each operation. Do not simply assign an arbitrary operation code to a particular operation: You will find that groups of operations have similar structures and require similar control signals such that a careful choice of the codes will simplify the logic for controlling the operations.

2.2 Requirements and constraints

1. You are permitted to modify only the Verilog file *arith_circuit.v*, the ROM contents file (*rom.txt*) and waveform files. You must not modify any other file or schematic in the project. You must not add any files, other than waveform files, to the project. Any additional modules that you might need to create to implement your design should be included in the *arith_circuit.v* file.
2. You are not permitted to modify the module or port declaration of the *arith_circuit* module.
3. Port declarations must use the ordered list port connection method. Dotted notation is not permitted.
4. You must implement your design using structural and dataflow Verilog constructs (gate primitives, assign statements with operators). **You may not use dataflow arithmetic operators** (e.g. `assign F = A + B;` or `assign F = A - B;`), **or any other operators listed on the “Other Dataflow Operators” slides.**
5. You are not permitted to create any schematics or to use behavioral Verilog constructs (e.g., case statements, for loops, always, generate).
6. Your design must be completely combinational.
7. You must consider the efficiency of the design and make design choices to minimize FPGA resource usage (i.e. transistors).
8. You must assign a unique 3-bit code to each operation.
9. Your *rom.txt* file must include the last four digits of your student ID number and the validation data described in the Completion section below.

2.3 Simulation

Verify the behavior of your circuit by simulating it in Quartus. Be sure to test multiple operand values for each operation. You will need to include your simulations in your submission (see the Completion section). You should carefully consider which sets of inputs are used to test your function unit so that a wide variety of cases are covered. You may need multiple screenshots to illustrate appropriate testing. Your input signals should be `SW[9:7]`, `SW[5:3]`, and `SW[2:0]`. Your output signals should be `LED[7:0]`, and if you would like to see the selected operand values, `A[7:0]` and `B[7:0]`. *Note that A and B are **output** signals, **not input** signals.* Be sure the signal names are readable; otherwise the waveform is meaningless.

Tip: A and B are **not** input ports; they are the output of the rom. You should not be assigning 8-bit values to A and B in your input waveform. You must select operand A and operand B using `SW[2:0]` and `SW[5:3]`, respectively. If you would like to see operand values in your simulation, you can add the output of the rom module using nodes A and B, as shown in Figure 2.1. Do **not** assign values to these signals in your input waveform. They are **outputs**.

Tip: To group just three bits of SW together in your simulation, first ungroup all of SW, then group the three bits of interest, e.g. `SW[9:7]`, `SW[5:3]`, `SW[2:0]`. This will make your simulation easier to read.

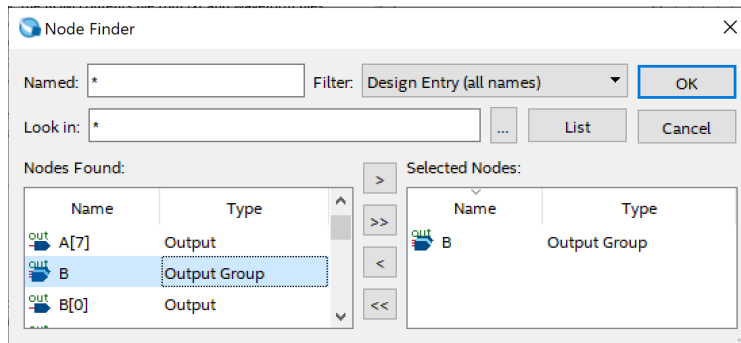


Figure 2.1. Selection of the A and B **output** nodes in the Node Finder window.

Tip: Don't forget to replace `-novopt` with `-voptargs=+acc`.

2.4 Testing

After your model simulates satisfactorily, you must compile it and then program the DE10-Lite board with it. Please refer to the lab manual for full instructions. Briefly, to compile your design, double-click **Compile Design** in the Tasks window, or select **Processing->Start Compilation** from the toolbar. When the project is successfully compiled, connect the board to your computer using the USB cable provided with the board. Then double-click on **Program Device (Open Programmer)** in the Tasks window, or select **Tools->Programmer** from the toolbar. The programmer window should open. Click on the **Start** button. The start button will gray out, and the LEDs on the DE10-Lite board will dim. In a few seconds, the Progress indicator on the programmer window should turn green and say 100% (Successful). Your design has now been downloaded to the board.

Tip: If you make changes to the operands in the `rom.txt` file, you must recompile in order to see the changes.

Test your circuit by using the switches on the DE10-Lite board to provide valid input combinations to the arithmetic circuit that you implemented on the FPGA. A switch provides logic-1 when it is in the up position and logic-0 when it is in the down position. Use SW[9:7] to apply each 3-bit code that you assigned to each operation. Use SW[2:0] to choose the A operand from the eight operands contained in `rom.txt`. Use SW[5:3] to choose the B operand.

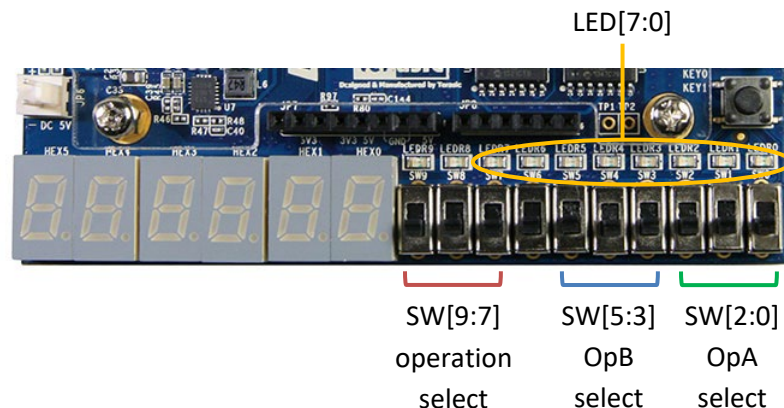


Figure 2.2. DE10-Lite board switches used for input and LEDs used for output.

Tip: The seven segment displays are not being used for this assignment. Because their pins are floating, you may see spurious activity due to activity on nearby pins. This is not relevant to this assignment and can be disregarded.

Tip: When you do a full compile, you will see a warning that informs you there is no exact pin location assignment(s) for 16 pins of 33 total pins. The A[7:0] and B[7:0] output ports were added to the project so that you would be able to view the operand values when you simulate. Because these were added to facilitate simulation, they are not actually connected to the I/O on the board. This is fine.

2.5 Cost and Speed Calculations

Using your final arithmetic circuit design, **as implemented in arith_circuit.v**, determine the number of transistors utilized. As part of your submission, you will need to include the transistor count as well as a tabular explanation of your calculations. It must be clear how you arrived at the numbers you present. You must clearly state any assumptions made. (For example, how did you assume conditional statements were implemented?) Submissions that utilize faulty assumptions or calculations will result in deductions. Be sure to follow the guidance provided by your instructor and in the lab manual on expectations for transistor counts.

Tip: If you implemented your circuit in Verilog using ANDs and ORs, you cannot assume it is implemented using NANDs. That is what “as implemented” means.

Recall: CMOS means complementary pairs of n-type (NMOS) and p-type (PMOS) MOSFETs. All circuits in this assignment and course must be implemented using CMOS processes. Disregarding this requirement will result in deductions. You must understand how logic gates are implemented in CMOS in order to correctly determine your transistor count.

You are not required to submit propagation delay calculations with this assignment. However, you *will* be using the arithmetic circuit designed here to build your entire function unit in Learning Experience C.2, and you will be required to submit propagation delay calculations with LE C.2, as well as an explanation of your calculations and any assumptions. Determining transistor count and propagation delay is much easier if you do not wait until you complete your design. You should keep track of the transistor count and propagation delay as you go along, for each subcircuit in your design. To prepare for LE C.2, you can determine the propagation delay of your arithmetic circuit now. You may use the primitive gate propagation delays assumed in previous assignments, or treat them as variables at this point.

Tip: Organizing your transistor count and propagation delay information into a spreadsheet now will save you a lot of time later. When cost and speed calculations are required for Learning Experiences, it is not sufficient to simply provide a number. It must be clear how you arrived at that number. You must understand which gates you used in your circuit, and how CMOS transistors are used to build each of those gates. If your tables are well-organized, you will be able to easily modify them when you add on to your circuit, instead of starting over for every Learning Experience. **See the lab manual.**

3. Completion

3.1 Documentation file

The documentation requirements for this assignment are minimal. You should submit a single pdf or doc/x file containing the following items in the order listed below.

- An evaluation of your implementation. This must include a transistor count calculation. (See Section 2.5.) Designs that utilize excessive FPGA chip resources (transistors) will result in deductions. Please note that this section will comprise a nontrivial portion of your grade for this assignment.

- Simulation waveforms demonstrating correct operation of your arithmetic circuit for *multiple operand values for each operation*. You should carefully consider which sets of inputs provide a reasonable test of the functionality of each operation. For example, did you make sure that your addition operation works for two positive numbers, two negative numbers, one of each? See Section 2.3.

3.2 Modify the rom.txt file for validation

Once you are confident that your design is working on the DE10-Lite board, you must edit the `rom.txt` file to contain the data shown in Table 3.1 before creating your project archive. Review the explanation of `rom.txt` in section 1.2. Remember that each 8-bit value stored in the rom must be expressed in hex in `rom.txt`.

Memory locations 0 and 1 must contain the BCD code for the last four digits of your VT ID number. Four digits will require four 4-bit codes, for a total of 16 bits. Since each location in the rom holds eight bits, the first eight bits should go in memory location 0; the second eight bits in memory location 1. The remaining six locations should contain the values listed in Table 3.1 below.

Tip: Repeat the examples below using your ID number. If you have questions about BCD encoding of your ID number or rom requirements, see a member of the 2544 course team **before** you submit.

Example 3.1: If your ID number is 905551234, the last four digits are 1234. The BCD code that represents 1234 is 0001001000110100. The first eight bits, 0b00010010 (0x12), should go in location 0; the second eight bits, 0b00110100 (0x34) should go in location 1.

Example 3.2: If your ID number is 905559876, the last four digits are 9876. The BCD code that represents 9876 is 1001100001110110. The first eight bits, 0b10011000 (0x98), should go in location 0; the second eight bits, 0b01110110 (0x76), should go in location 1.

The contents of `rom.txt` is expressed in hex; therefore, for these two examples, `rom.txt` should contain the values shown in the last two columns of Table 3.1.

Table 3.1 ROM contents of addresses 0-7 to be used for validation.

ROM address	Contents in binary 0b	Contents in hex 0x	How rom.txt should look using the example ID 1234	How rom.txt should look using the example ID 9876
0	1 st 8 bits of the BCD code for the last four digits of your ID	Depends on ID	@0 12 34 E8 91 7F 55 06 80	@0 98 76 E8 91 7F 55 06 80
1	2 nd 8 bits of the BCD code for the last four digits of your ID	Depends on ID		
2	11101000	E8		
3	10010001	91		
4	01111111	7F		
5	01010101	55		
6	00000110	06		
7	10000000	80		

Tip: Editing all 8 locations in `rom.txt` is **required** for submission and validation. Do not skip it.

3.3 Circuit Validation

Following submission, your circuit will be validated by 2544 course staff. You will provide the source files that will allow us to compile the same files that you used to implement your arithmetic circuit on the DE10-Lite Board. Instructions for creating a Quartus archive (*.qar file) for submission can be found in the lab manual. When you create the archive, you can choose the filename and where it will be stored. Make sure you name the qar file appropriately; you do not want to confuse it with the original qar file provided with the project. Upload the qar file to Canvas, **making certain that you upload the completed archive that contains your work, and not the one that was provided to you.** The file that you submit is the file that will be graded.

Tip: Be sure to choose a filename that will allow you to differentiate between your new archive and the one that was provided to you. (See the lab manual and the help video on the ECE 2544 website under Supplemental Material: Hardware and Software.)

Tip: You must edit rom.txt to contain the required validation data before creating the archive. Otherwise, your archive will not be correct.

Complete the validation sheet. You will need to enter the last four digits of your ID, and indicate which operations you implemented and the 3-bit code assigned to each operation.

Tip: Notice that mova appears in the validation table twice. This does NOT mean you should assign two different 3-bit codes to the mova operation. Enter the same 3-bit code for all occurrences of mova.

Tip: We will validate your circuit after you submit. It is good engineering practice to verify that the validation procedure, as shown on the validation sheet, produces correct results, by testing it on your board before you submit it. Your validation results should not be a surprise.

Tip: If you do not complete the required information on the validation sheet, we will not be able to validate your circuit and your validation score will be zero.

3.4 Submission Requirements

Submit the following files on Canvas. **Do not put them in a zip file!**

1. Your Archived Project **LE_C1a_YourPID.QAR**. This must contain the correct submission rom.txt (See Table 3.1).
2. Your completed Verilog file **arith_circuit.v**. This must match the .v file in your archive.
3. Your modified memory file **rom.txt**. This must match the rom.txt file in your archive. (See Table 3.1)
4. Required documentation in a single pdf or doc/x file (see above)
5. Your validation sheet. Remember to complete the table listing your operation set and the 3-bit code assigned to each operation.

Tip: You are required submit your qar and your .v and rom.txt file. The purpose for this requirement is to facilitate efficient grading. Failure to submit any of the required files will result in point deductions.

Tip: Your PID is the text of your VT email address before "@vt.edu". It is NOT your ID number, which starts with "90..."

3.5 Grading

The design project will be graded on a 100 point basis, as shown in the Canvas Assignment rubric.